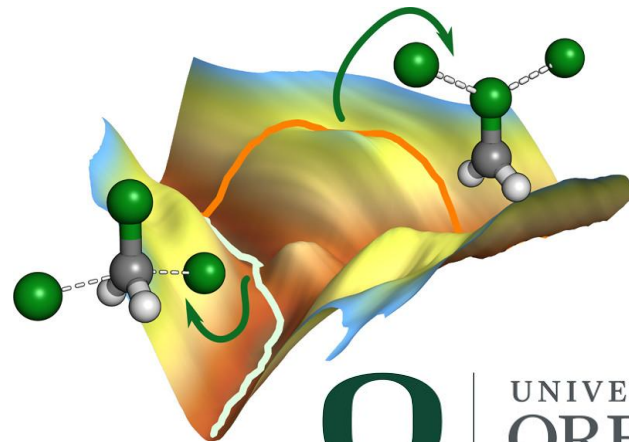
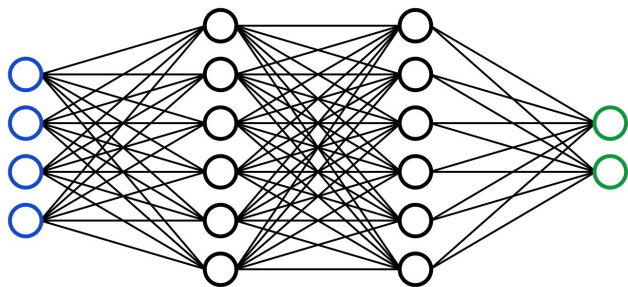


Machine Learning for Chemistry

Winter 2025

Advanced Topic 3: Generative Machine Learning



UNIVERSITY OF
OREGON

Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

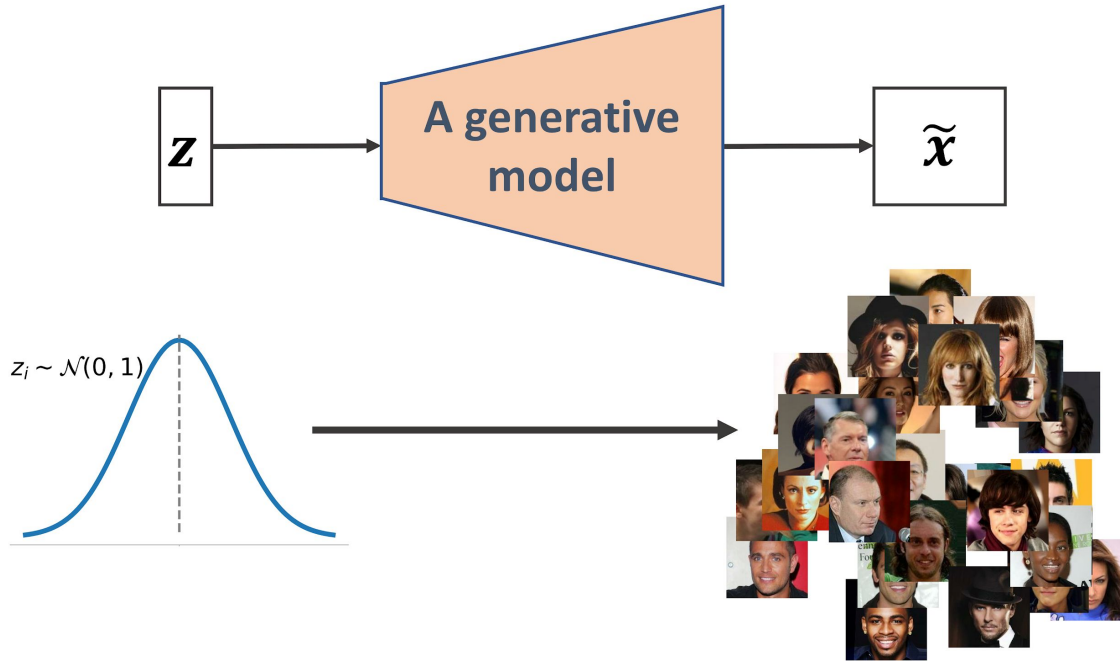
Overview of Topics

- Variational Autoencoder (VAE)
- Generative Adversarial Network (GAN)
- Normalizing Flow
- Diffusion Model

References:

1. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 17)

Generative Machine Learning

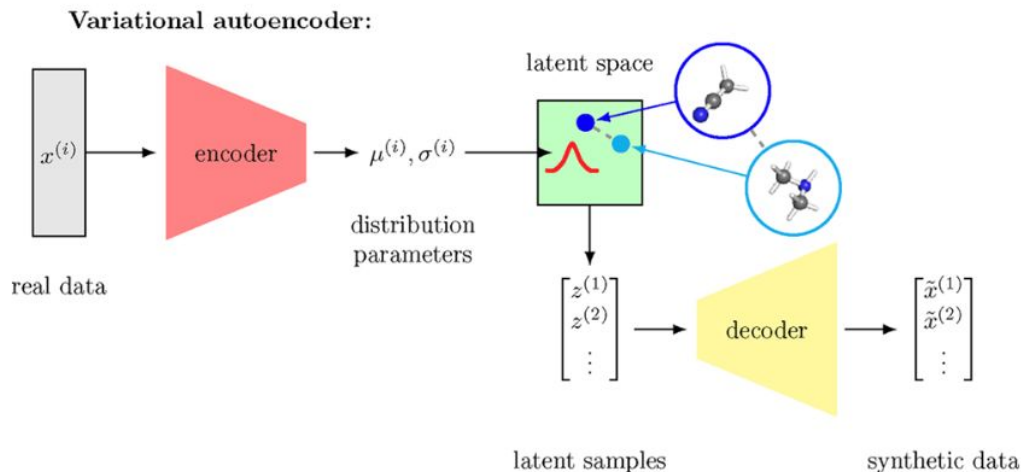
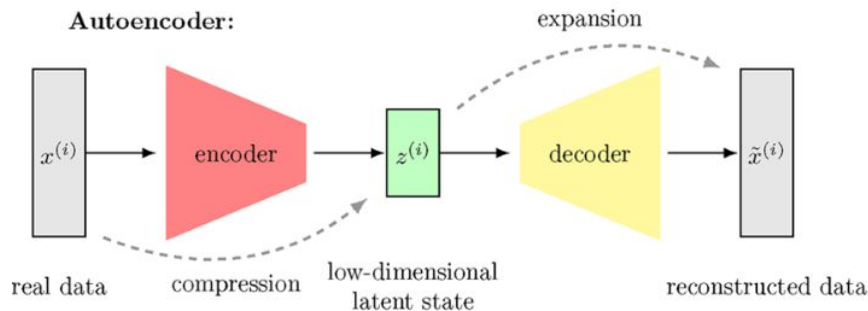


Generate new data by learning patterns from
already available data



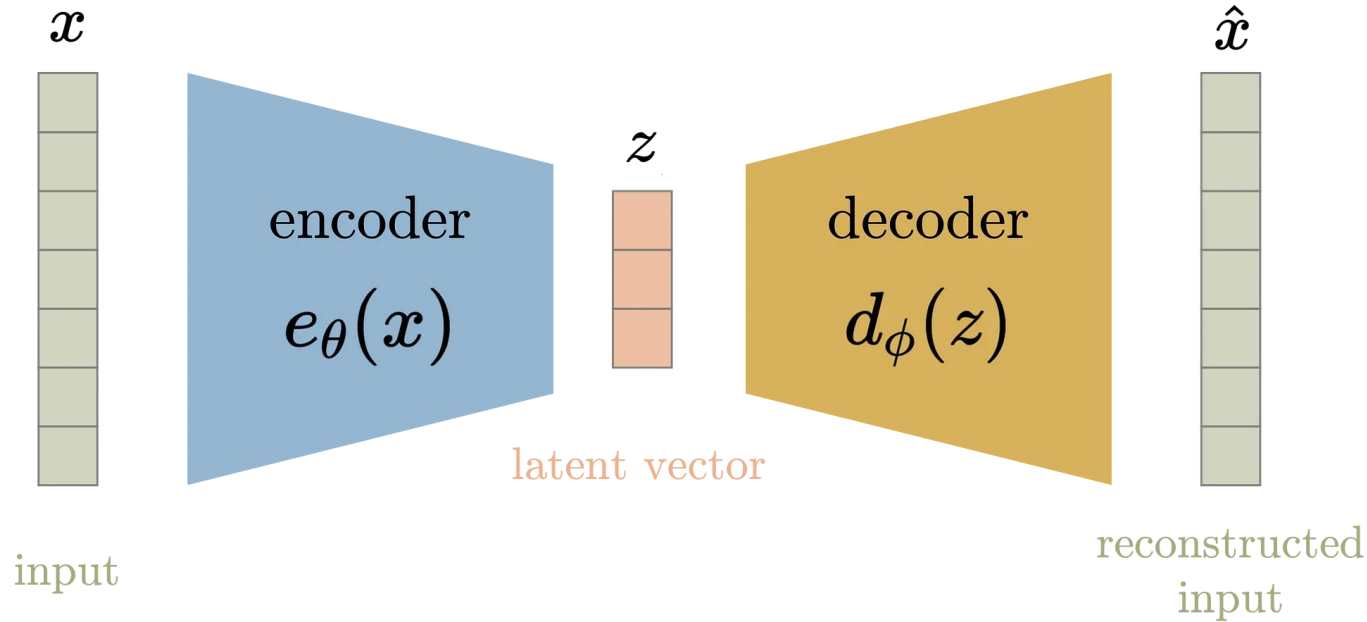
**Can be catastrophic if not
done correctly**

Variational Autoencoder (VAE)



Janet, Jon Paul,
and Heather J.
Kulik. *Machine
Learning in
Chemistry*. Vol. 1.
American
Chemical Society,
2020.

Standard Autoencoder



$$loss = \|x - \hat{x}\|_2 = \|x - d_\phi(z)\|_2 = \|x - d_\phi(e_\theta(x))\|_2$$

Figure courtesy: Aqeel Anwar. Towards Data Science 2021

Variational Autoencoder (VAE)

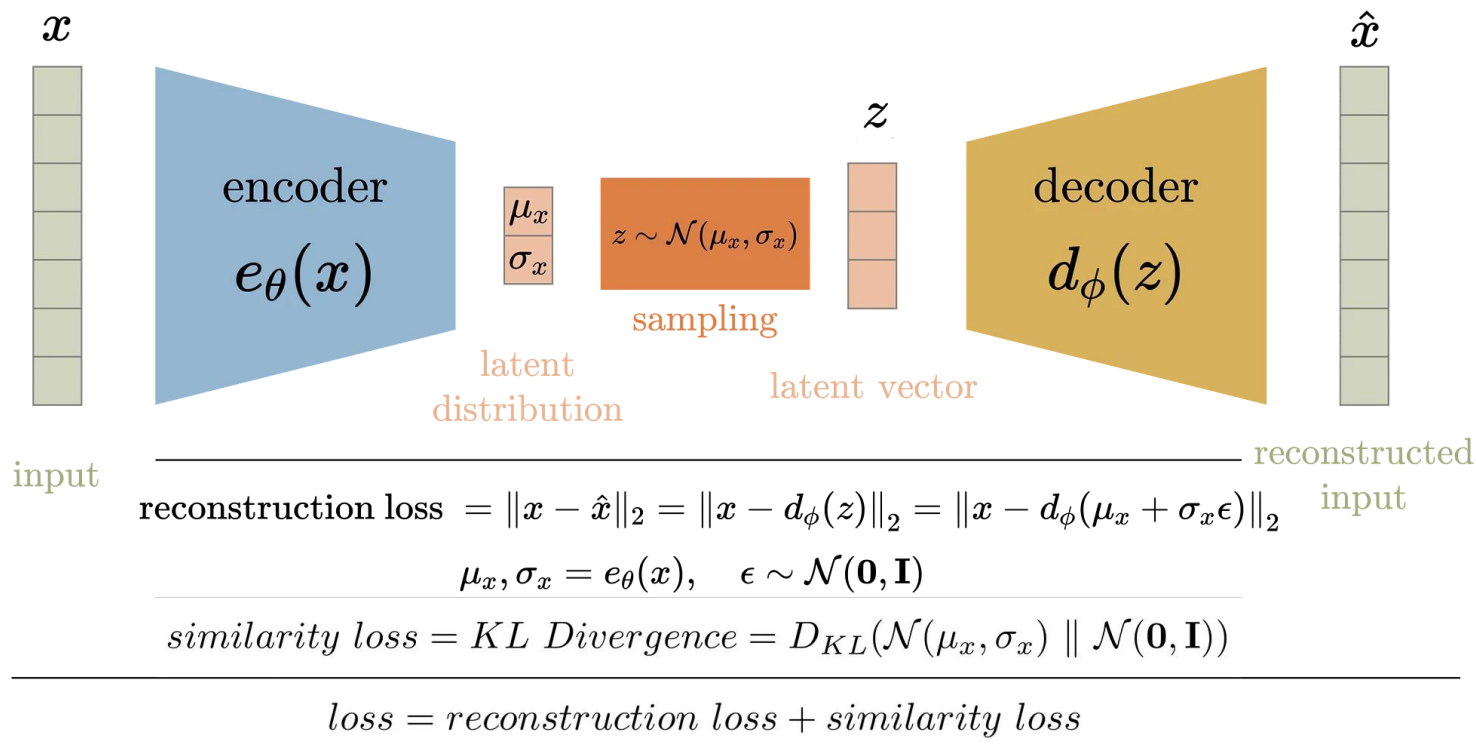
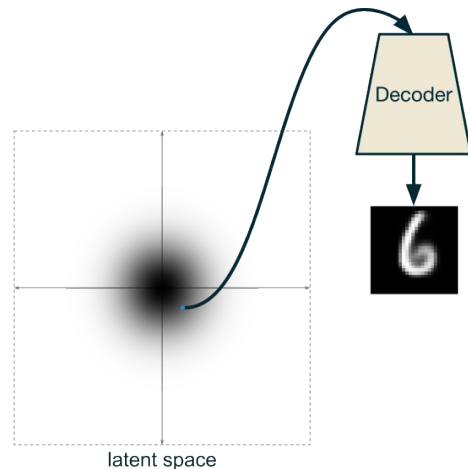
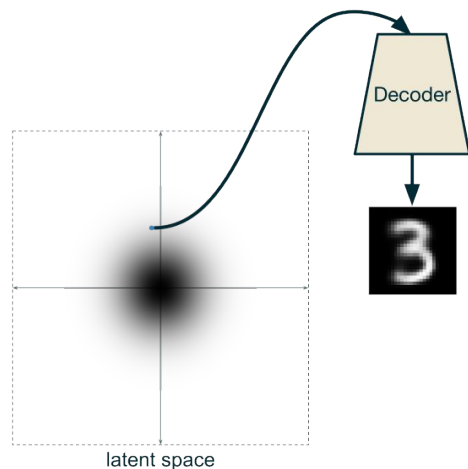
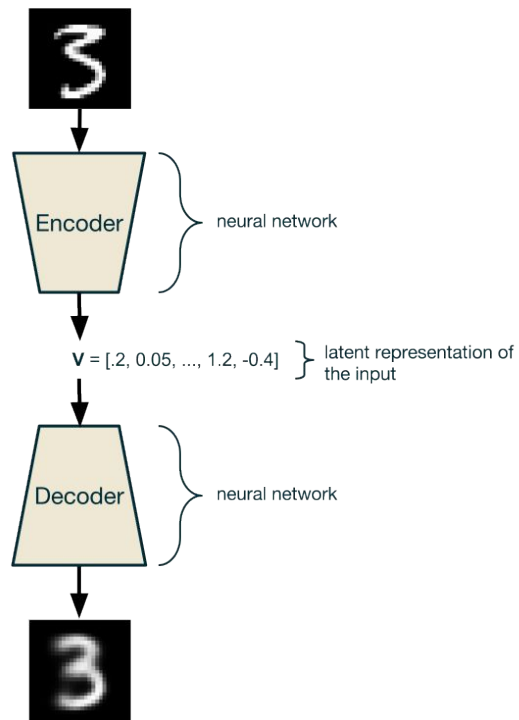


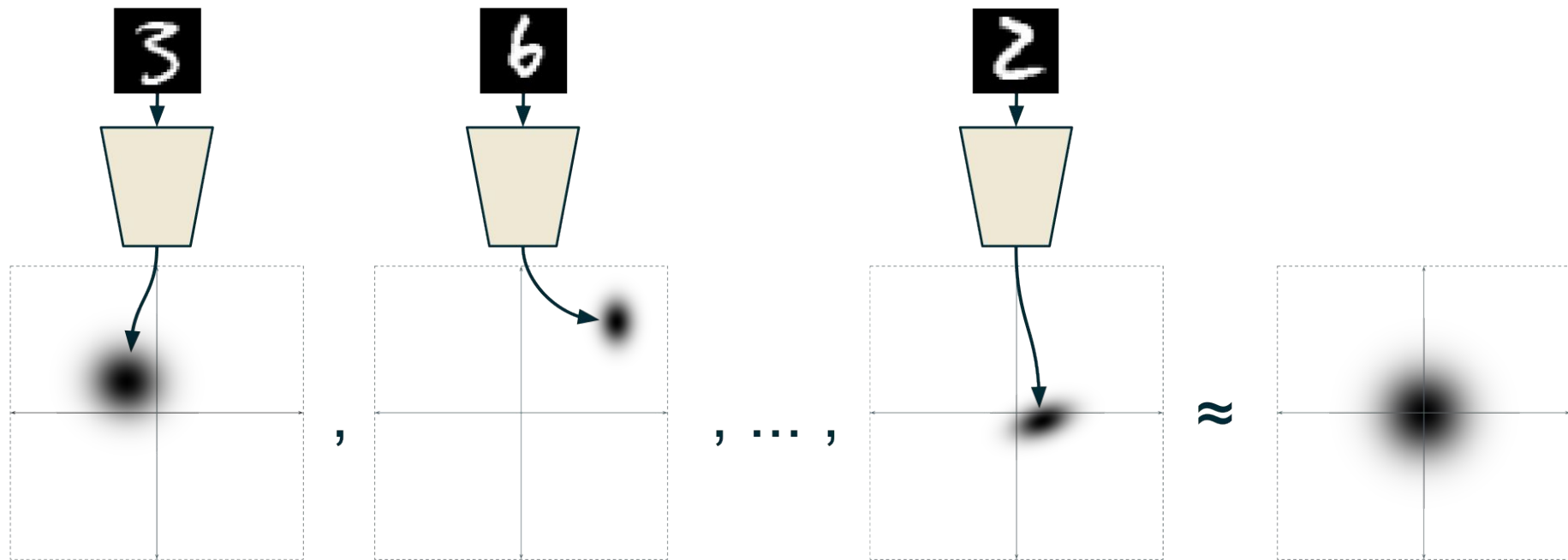
Figure courtesy: Aqeel Anwar. Towards Data Science 2021

Variational Autoencoder (VAE)



Courtesy: Isaac Dykemann 2016

Latent Space



Courtesy: Isaac Dykemann 2016

Training a Variational Autoencoder (VAE)

Loss function

The loss function of VAE has two objectives

1. Reconstruct the input
2. Latent space should be normally distributed

The reconstruction error, just like in Autoencoder, is the mean squared loss of the input and reconstructed output. The similarity loss is the **Kullback–Leibler (KL) divergence** between the latent space distribution and standard gaussian (zero mean and unit variance). The loss function is the sum of these two losses.

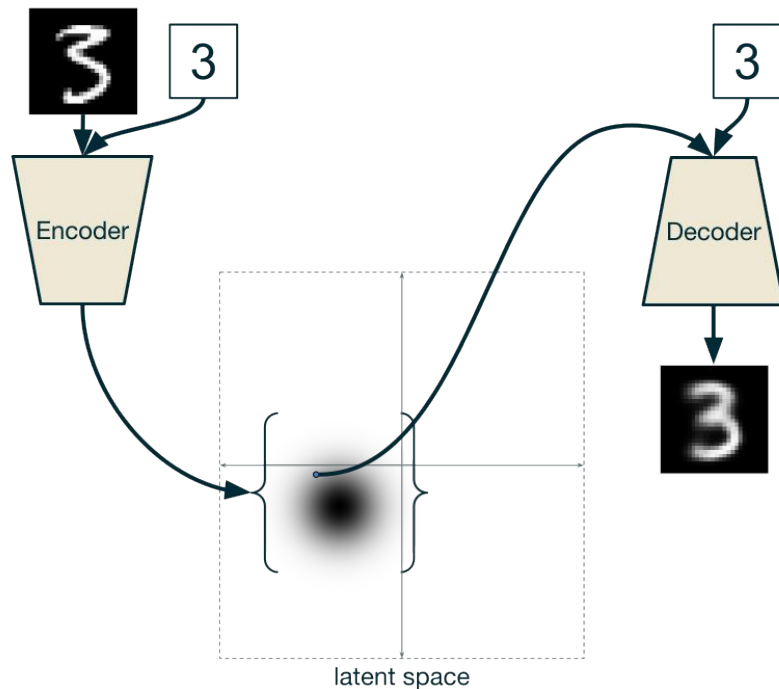
$$D_{\text{KL}}(P \parallel Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

Conditional Variational Autoencoder (CVAE)

The decoder of VAE can produce reasonable handwritten digit images. However, the decoder **cannot produce an image of a particular number** on demand.

Conditional Variational Autoencoder: an extra input to both the encoder and the decoder. At training time, the number whose image is being fed in is provided to the encoder and decoder.

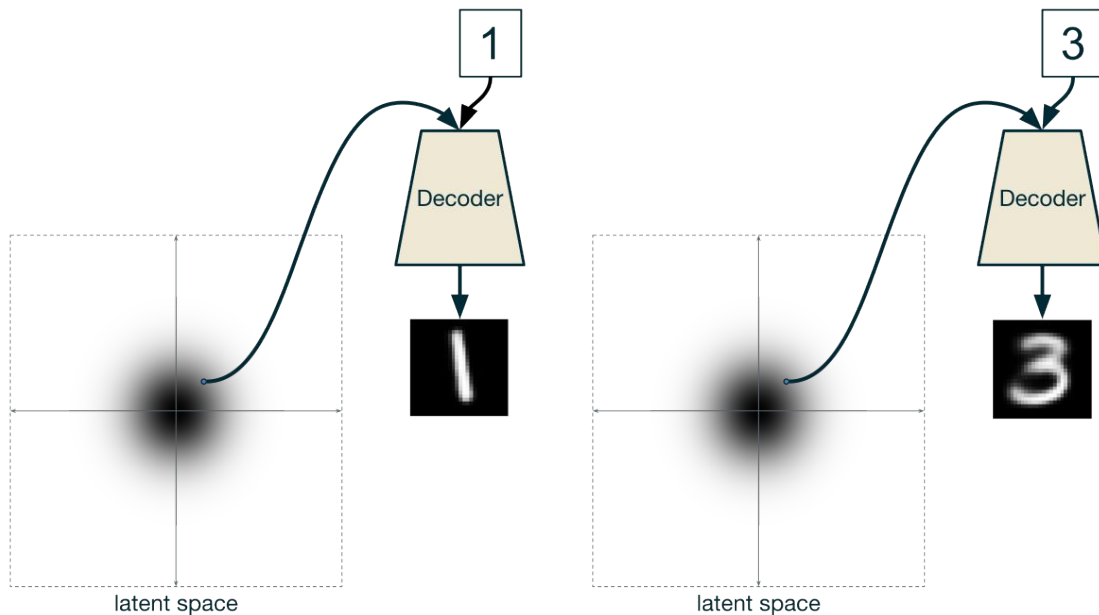
To generate an image of a particular number, just feed that number into the decoder along with a random point in the latent space sampled from a standard normal distribution.



Courtesy: Isaac Dykemann 2016

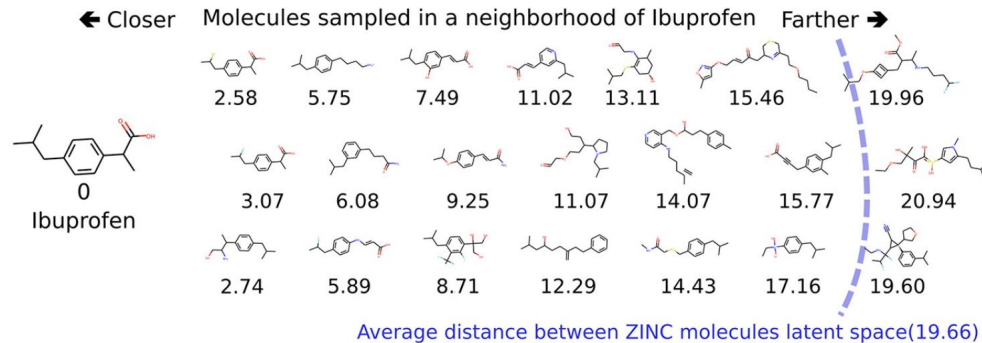
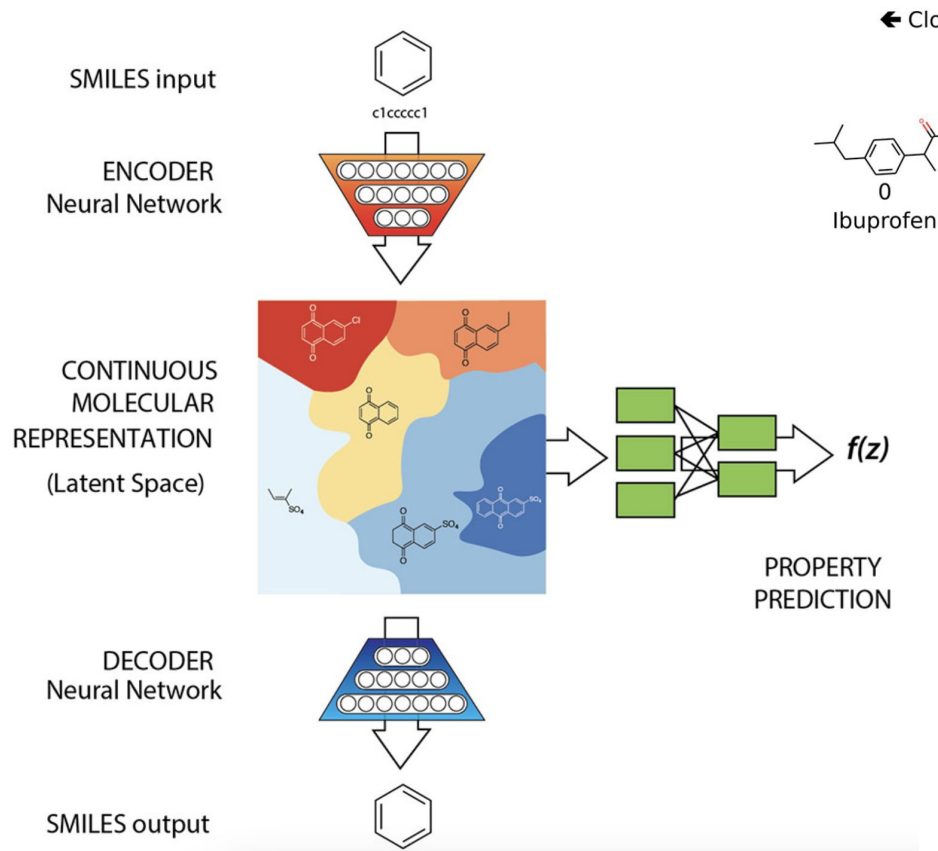
Conditional Variational Autoencoder (CVAE)

Even if the same point is fed in to produce two different numbers, the process will work correctly, since the system no longer relies on the latent space to encode what number you are dealing with. Instead, the latent space encodes other information, like stroke width or the angle at which the number is written.



Courtesy: Isaac Dykemann 2016

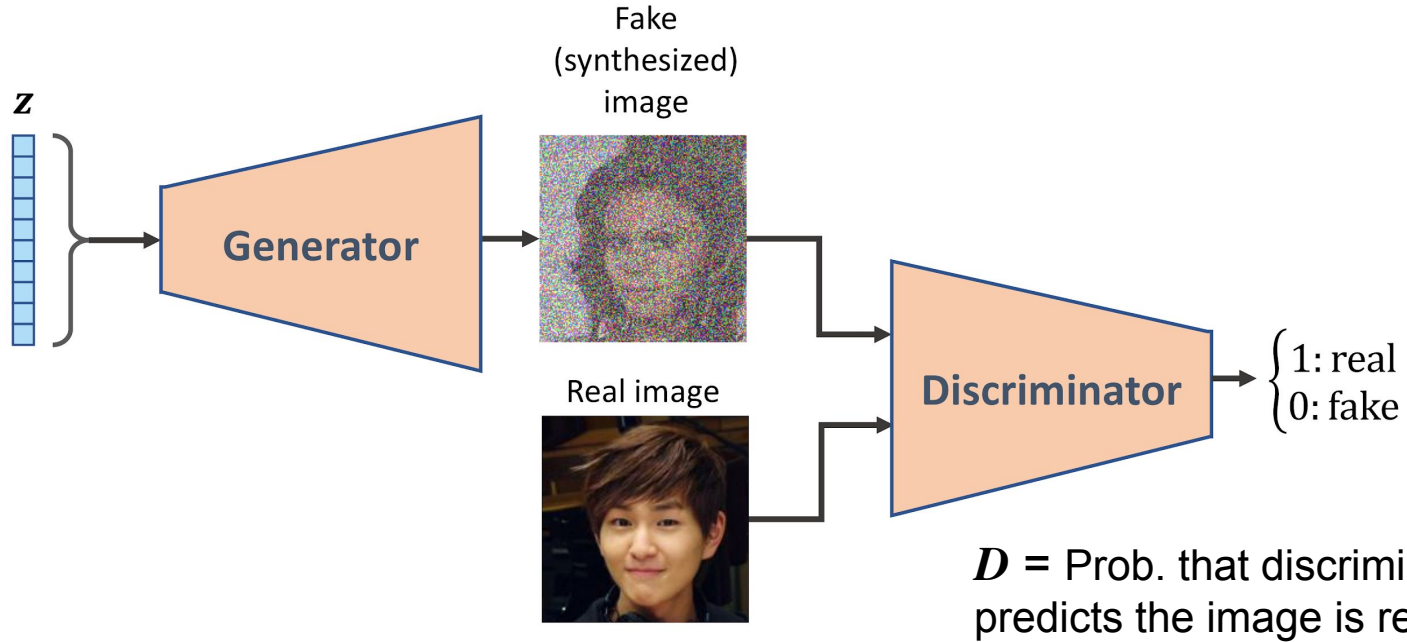
Variational Autoencoder for Molecule generation



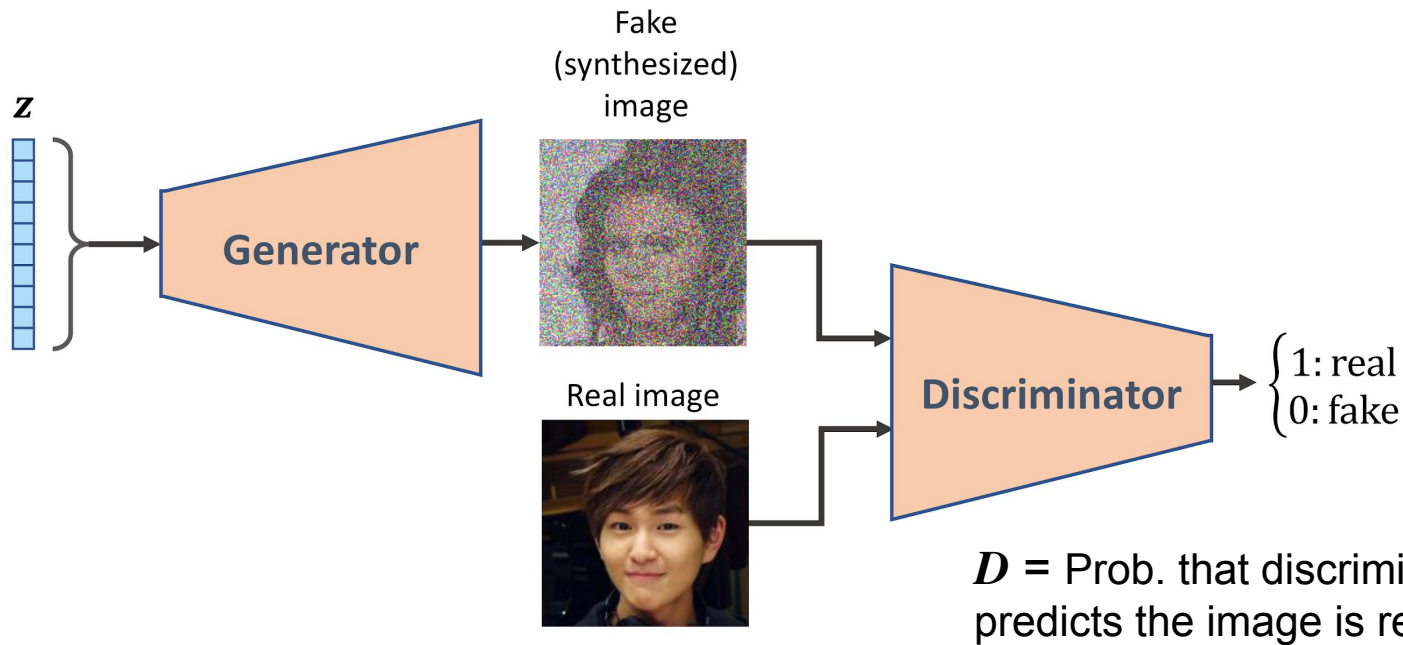
Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules

Gómez-Bombarelli et al. ACS Central Science 2018 4 (2), 268-276
DOI: 10.1021/acscentsci.7b00572

Generative Adversarial Network (GAN)

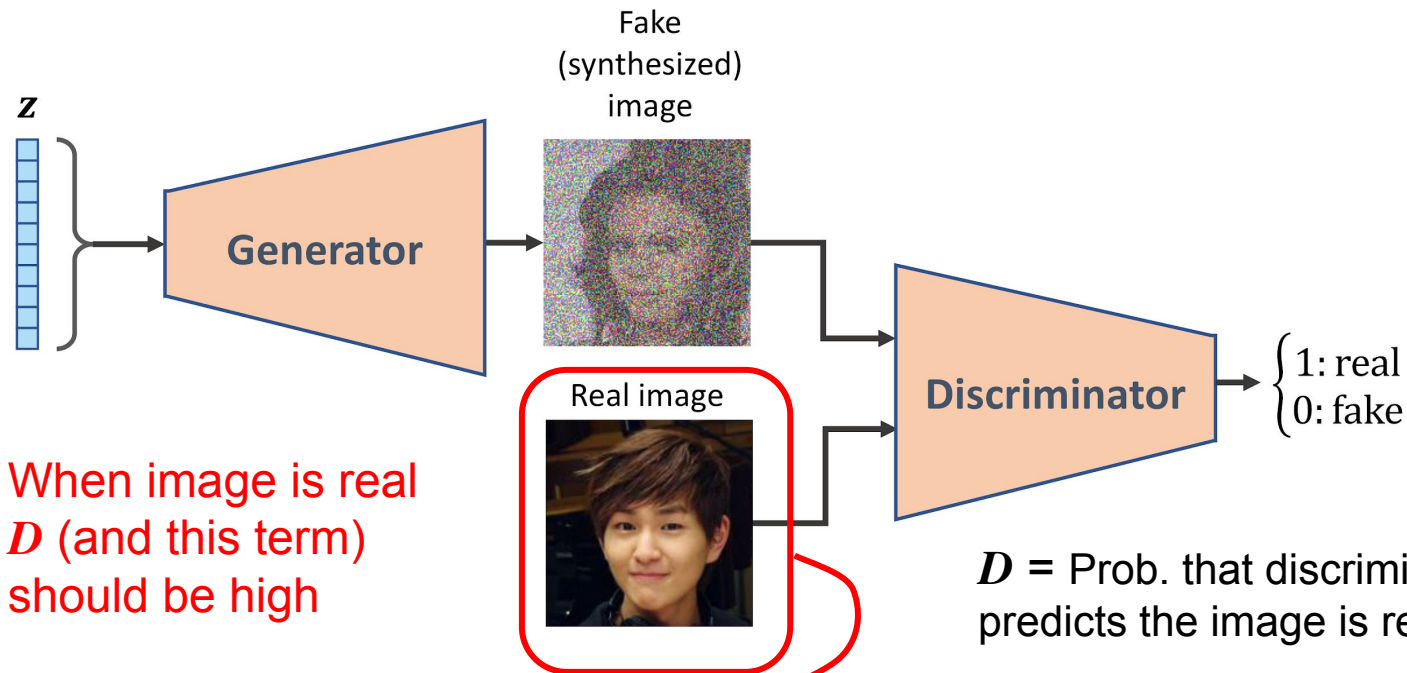


Generative Adversarial Network (GAN)



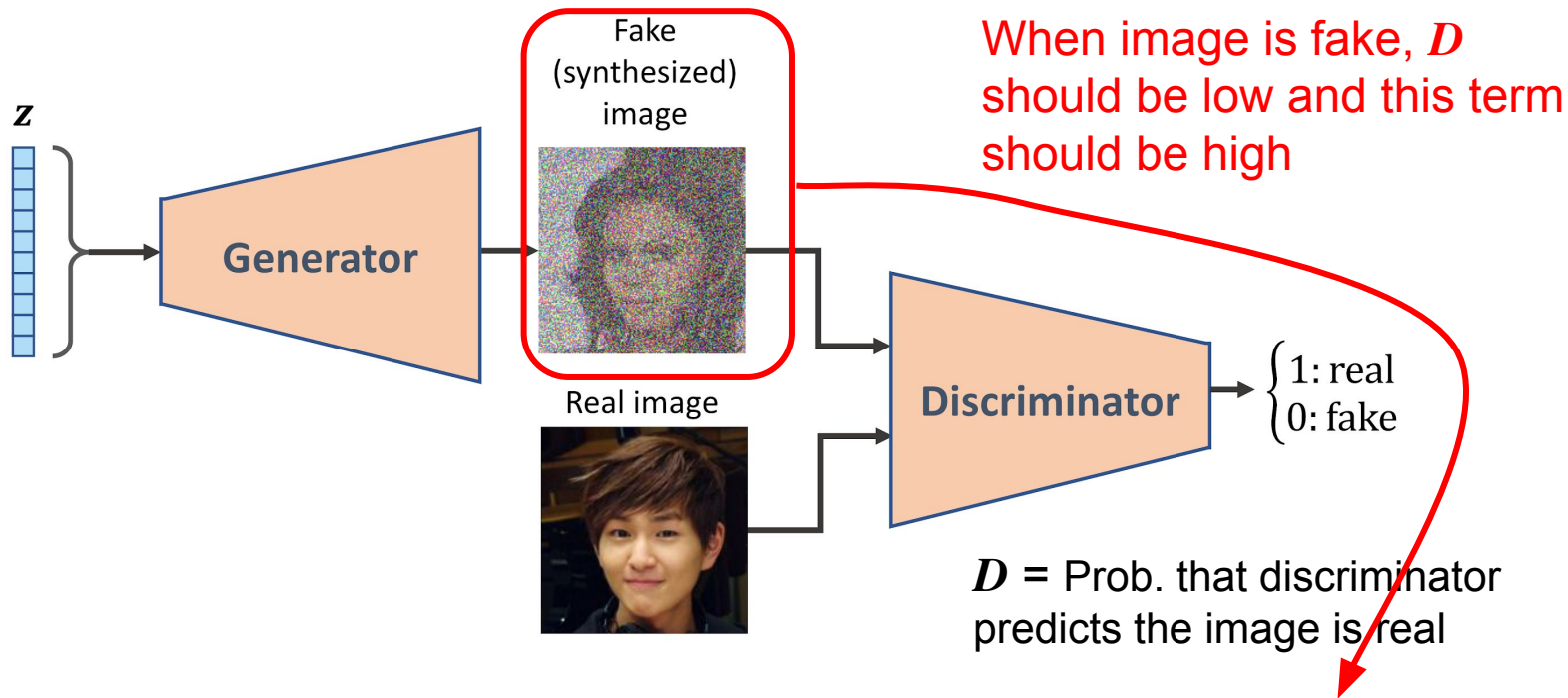
$$V(\theta^{(D)}, \theta^{(G)}) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

Generative Adversarial Network (GAN)



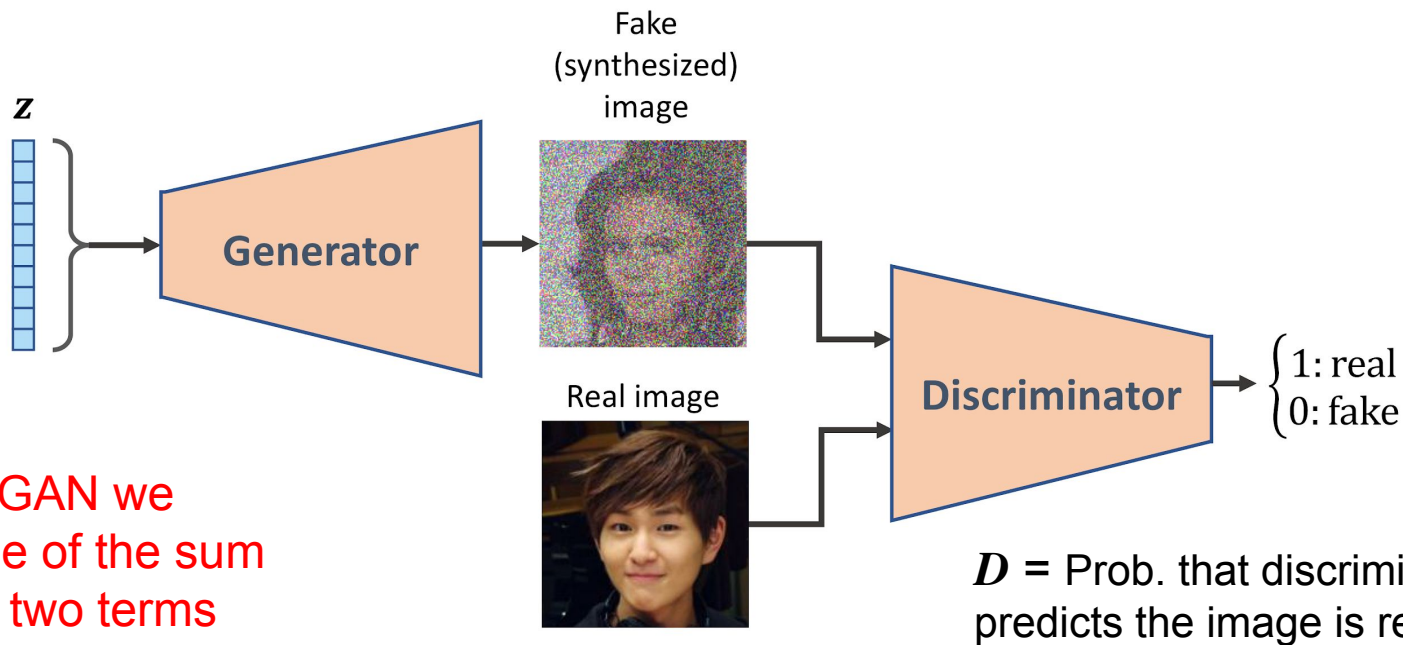
$$V(\theta^{(D)}, \theta^{(G)}) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

Generative Adversarial Network (GAN)



$$V(\theta^{(D)}, \theta^{(G)}) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

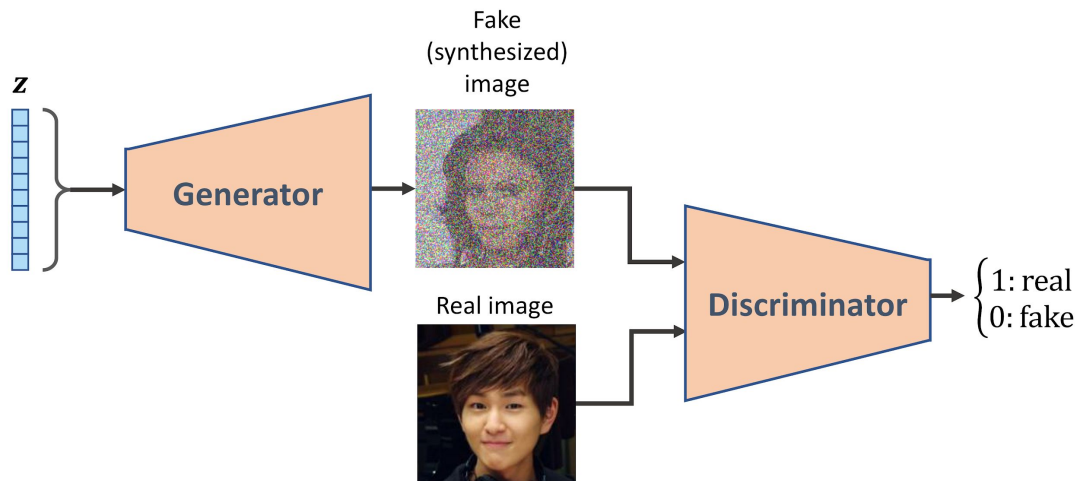
Generative Adversarial Network (GAN)



To train GAN we
maximise of the sum
of these two terms

$$V(\theta^{(D)}, \theta^{(G)}) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

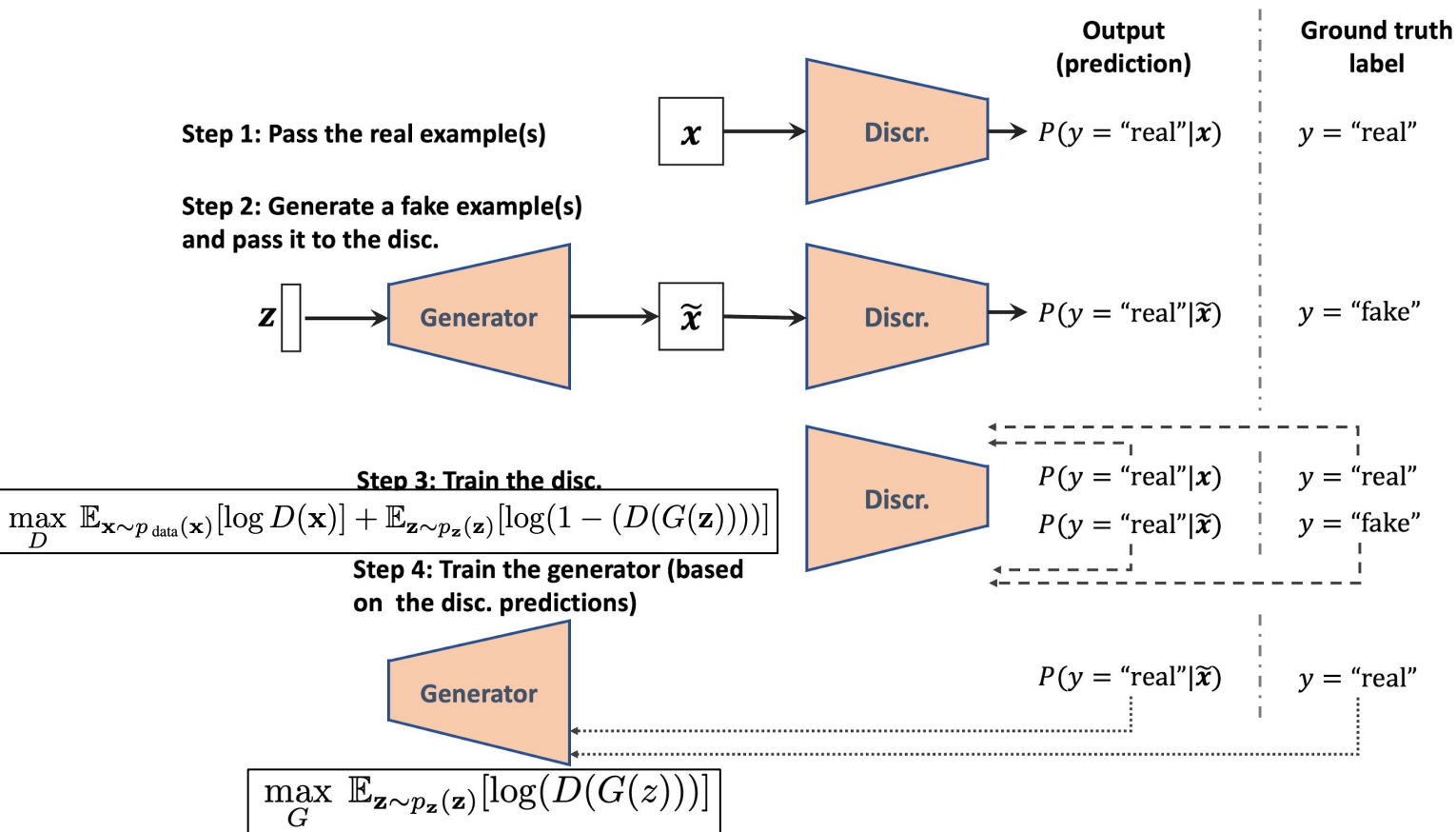
Generative Adversarial Network (GAN)



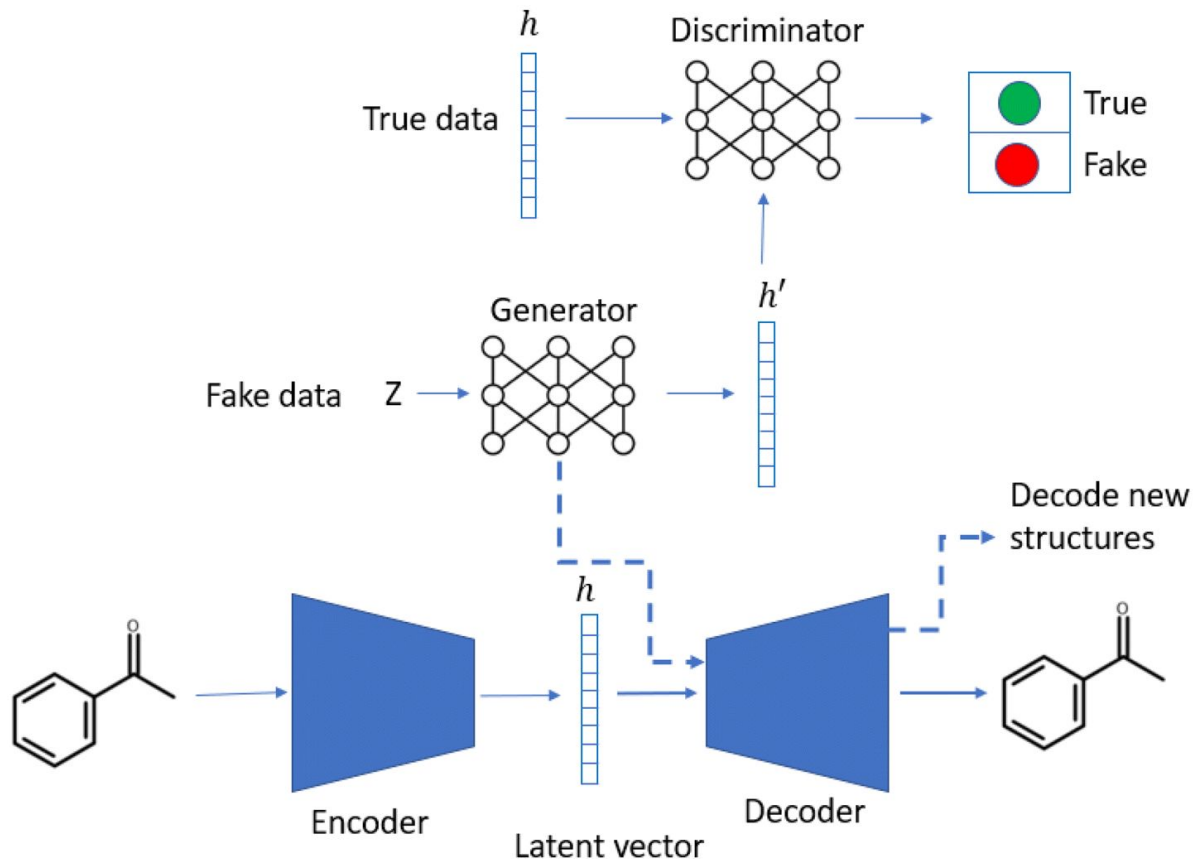
GAN can be thought of as a **competition between two neural networks**. The Generator network is trying to fool the Discriminator network and the Discriminator is trying to catch it.

At some point the generator will become so good in generating fake data that the discriminator can no longer distinguish. We can use that generator going forward.

Training a Generative Adversarial Network (GAN)



Application of GAN to Chemistry

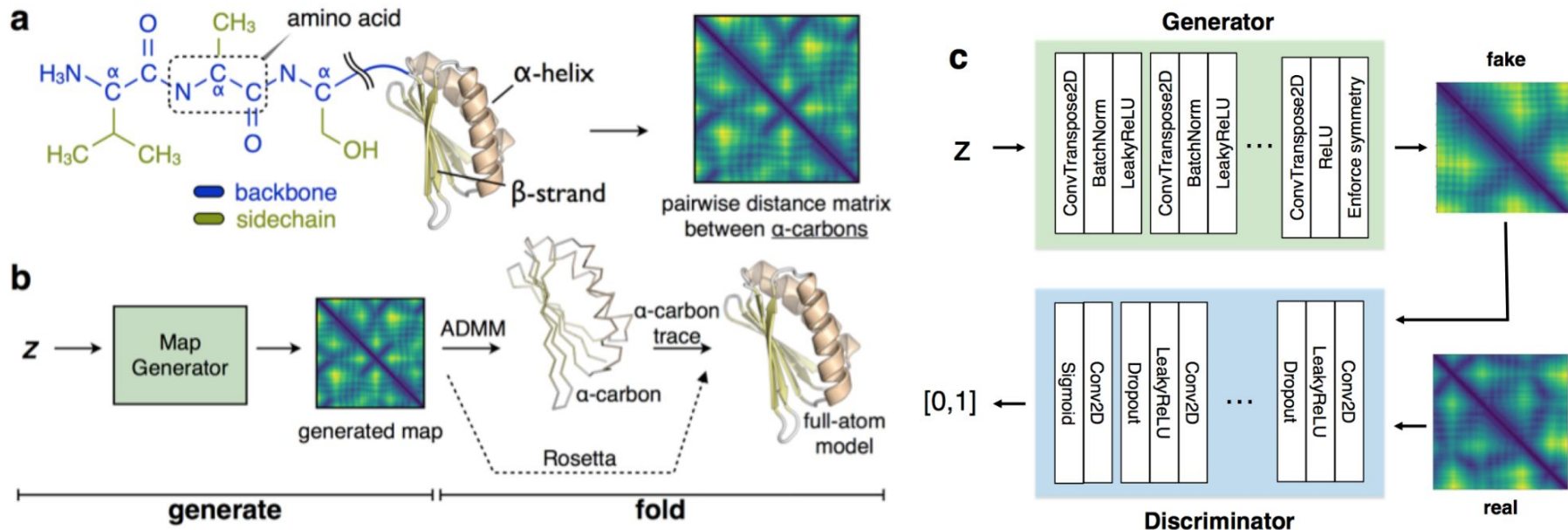


Prykhodko O, Johansson S. V, Kotsias P.-C, Arus-Pous J, Bjerrum E. J, Engkvist O et al.

A de novo molecular generation method using latent vector based generative adversarial network.

Journal of Cheminformatics. 2019;11:74.

Application of GAN to Chemistry



Deep convolutional generative adversarial network (DCGANs)

Anand, Namrata, and Possu Huang. **Generative modeling for protein structures**. Advances in neural information processing systems 31 (2018).

Sampling from a probability distribution

Generative ML can also be thought of sampling from a high dimensional complex probability distribution.

For example, if I give you a protein sequence and ask you to generate possible folded structures, you need to sample from the Boltzmann distribution $\sim e^{-\beta U(x)}$. At a given temperature low energy structure will be more probable. So if we could directly generate random samples from Boltzmann distribution, we would be able to successfully find stable structures.

But for most molecules, it is notoriously difficult to even write a compact expression of the Boltzmann distribution. This is because the internal energy $U(x)$ is high dimensional and includes many bonded and non-bonded terms.

We need to transform the complex distribution to a simpler one before sampling from it.

Normalizing Flow

Normalizing Flow (and other flow based models) learns how to transform a complex distribution into a simple Gaussian distribution.

Then random numbers are sampled from the Gaussian distribution.

Then inverse transform these random numbers to convert them into real data.

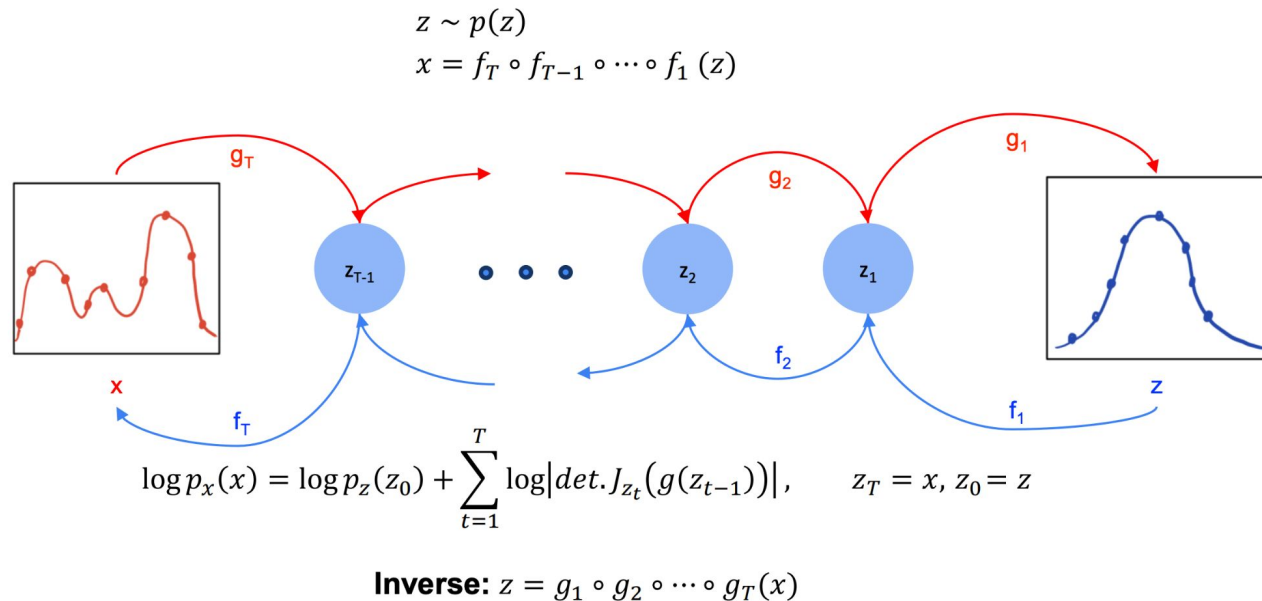


Figure courtesy: Abuzar Khan,
Carnegie Mellon University, 2023

Diffusion Model

Periodically add noise to convert original data (X_0) into complete Gaussian distribution. Learn the transformation backwards into original data.

Has to be done stepwise because learning the whole transformation using one network might be too difficult.

Now use the “reverse diffusion” to generate new data from random noise.

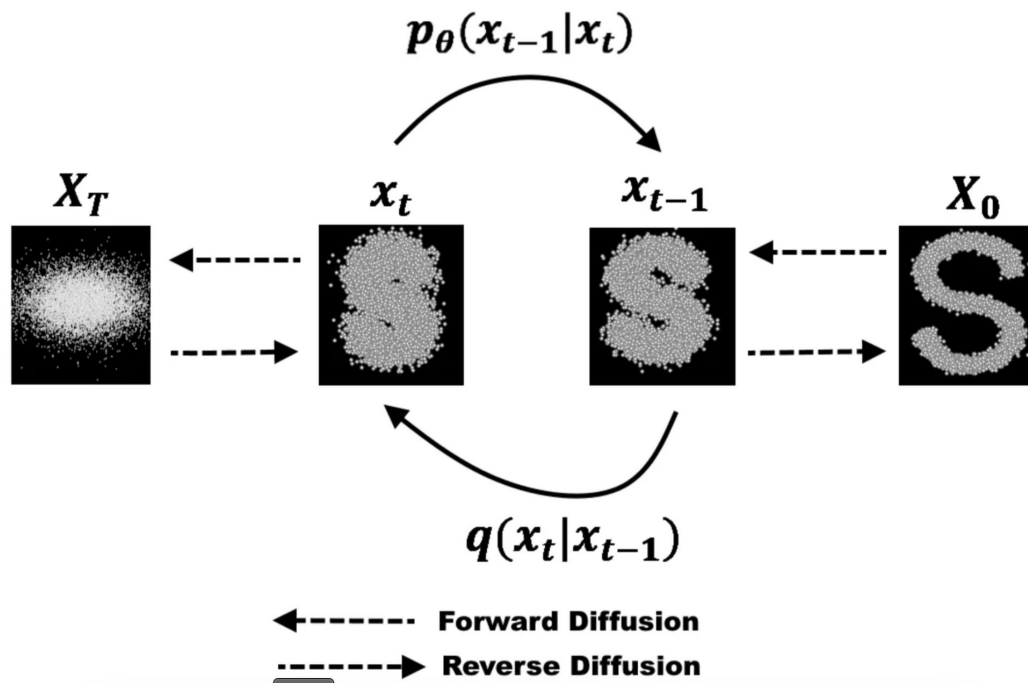
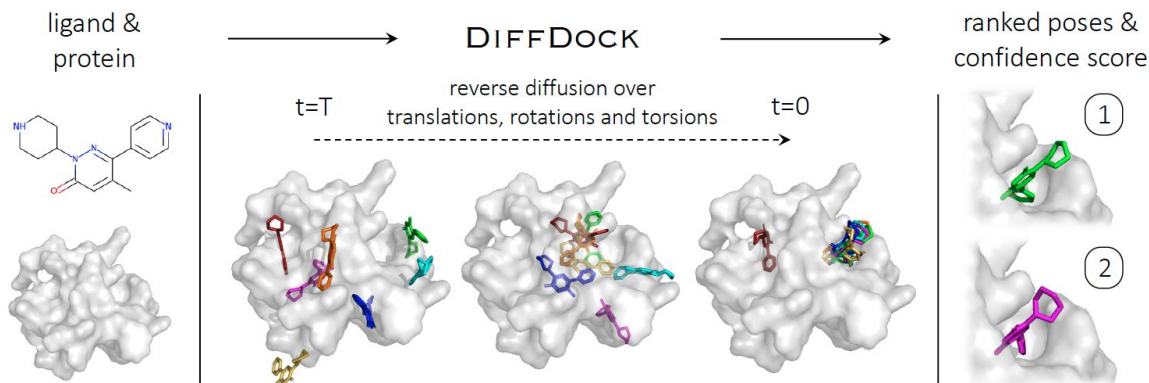


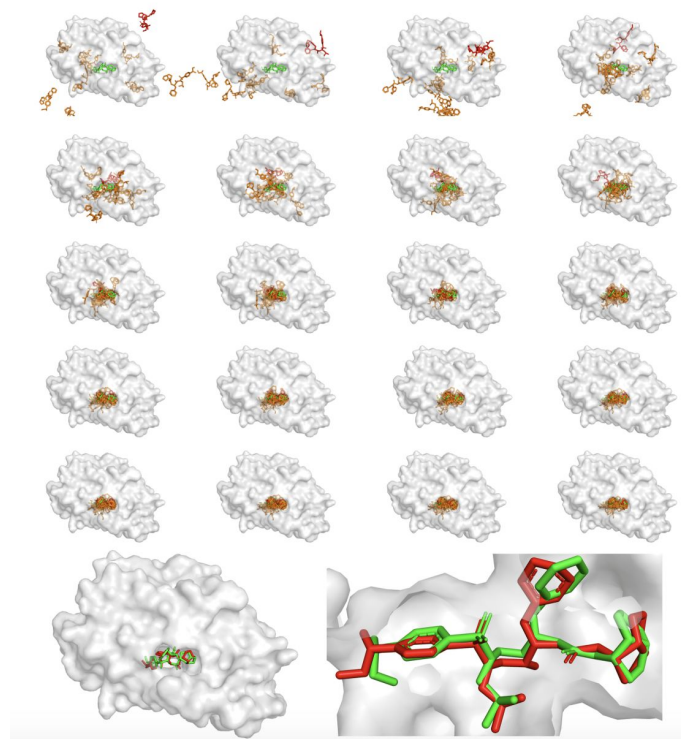
Figure courtesy: J R Siddiqui, 2022

Chemistry Example of Diffusion Model

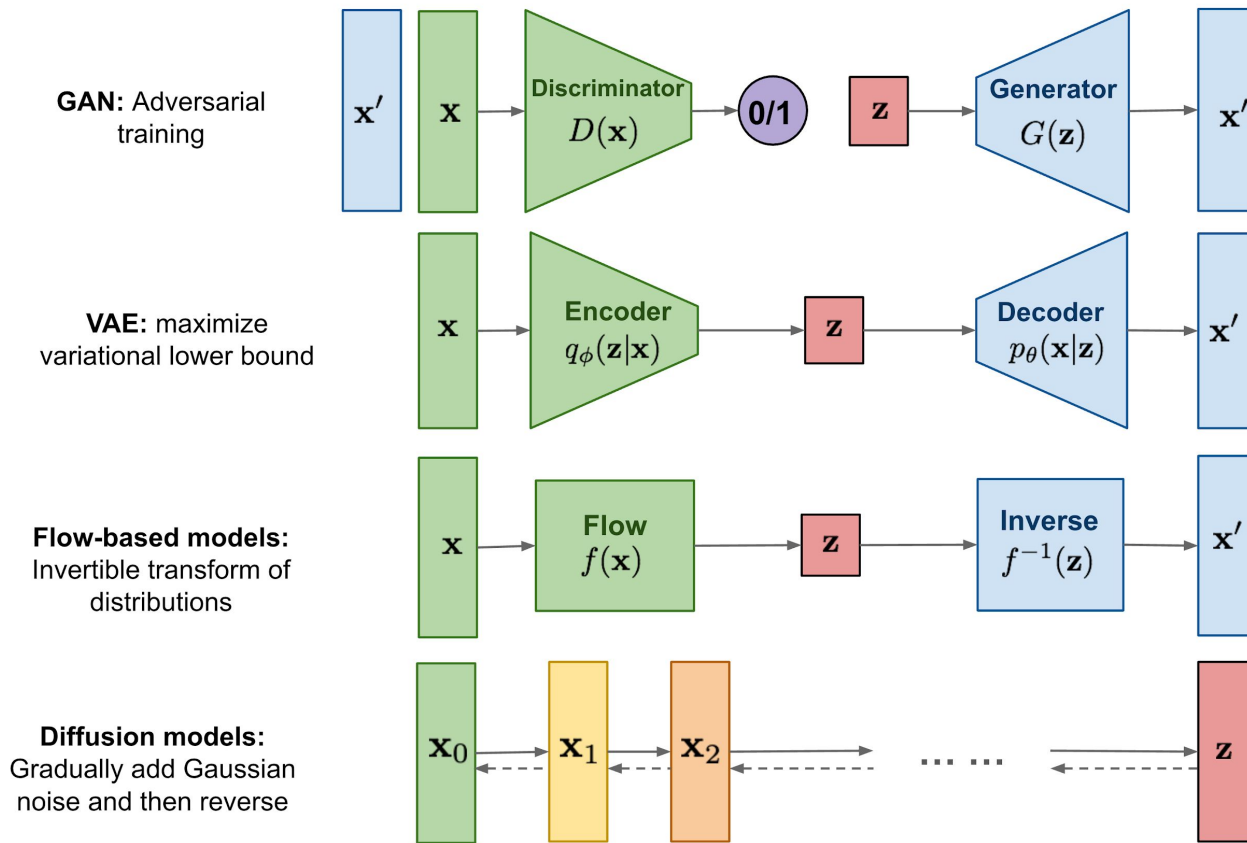


- Add noise to the crystal structure of a drug bound to a protein to produce random distribution of the ligand.
- Learn the transformation back to the bound state.
- Generated docked (bound) pose for unknown drugs

Corso, Gabriele, et al. "**Diffdock: Diffusion steps, twists, and turns for molecular docking.**" arXiv preprint arXiv:2210.01776 (2022).



Summary of Generative Models



Courtesy:
Lilian Weng,
2021